

MEMO-Appflow Beamer 逐页讲稿

讲解人: Jingyi Guo

组员: Mosha Huang Kewei Chen Chengyu Fan

2026-07-02

目录

使用说明	3
1 开场与系统总览	3
第 1 页: MEMO-Appflow	3
第 2 页: 汇报路线	3
第 3 页: 项目不是单纯的 app 推荐	3
第 4 页: 宏观闭环: 系统观察、模型决策、系统执行	3
第 5 页: 手机内运行边界	4
2 eBPF 采集与压缩	4
第 6 页: 为什么要引入 eBPF	4
第 7 页: 采集的 14 个 tracepoints	4
第 8 页: Pixel 5 上的 kernel/runtime 条件	4
第 9 页: tracepoint 到 counter key	5
第 10 页: 真实采集窗口长什么样	5
第 11 页: Android 端采集运行时: 代码结构概览	5
第 12 页: app 序列和 eBPF 的时间对齐	5
第 13 页: timeline window 示例	5
第 14 页: eventType 到 category: 权重矩阵	6
第 15 页: Category score 如何算	6
第 16 页: Context JSON 字段	6
第 17 页: 真实 Context JSON 摘录	6
第 18 页: 压缩不是丢证据, 而是换表达	6
3 MAPLE 推理与真实应用映射	7
第 19 页: MAPLE 三阶段	7
第 20 页: Stage 1 prompt: 预测用户可见 app 类别	7
第 21 页: Stage 1 真实输出	7
第 22 页: Stage 2 prompt: 类别到候选 App ID	7

第 23 页: Stage 3: scheduler_bits 动作协议	7
第 24 页: 小模型输出解析也要工程化	8
第 25 页: 从 MAPLE App ID 到真实 Top-3: 映射表怎么来	8
第 26 页: 从 MAPLE App ID 到真实 Top-3: 用户看到什么	8
第 27 页: ActionExecutor 执行的是真动作	8
第 28 页: 实时模式任务队列	9
第 29 页: Widget: 把预测变成可用的产品入口	9
4 实验设计与结果	9
第 30 页: 实验总览: 我们要回答什么	9
第 31 页: 实验之间的关系	9
第 32 页: 9 分钟实时实验: 动机与方案	9
第 33 页: 9 分钟实时实验: 结果	10
第 34 页: 9 分钟实验中的调度记录	10
第 35 页: 30 条真实记录预测实验: 实验设计	10
第 36 页: 30 条预测实验: 评估指标	10
第 37 页: 30 条预测实验: 单条指标怎么算	10
第 38 页: 理论消融: 先在可控数据上验证 eBPF 信号	11
第 39 页: 30 条真实记录: 最终预测结果	11
第 40 页: 单条样本的直观结果	11
第 41 页: 系统调度性能实验: Stage 3 是否有用	11
5 关键代码与实现落点	12
第 42 页: 关键代码: 三分钟采集窗口	12
第 43 页: 关键代码: 有界实时记忆	12
第 44 页: 关键代码: MAPLE 输入构造	12
第 45 页: 关键代码: 本地模型调用	12
第 46 页: 关键代码: 真实应用映射	12
第 47 页: 关键代码: 系统调度动作	13
第 48 页: 关键代码: 桌面 Widget	13
第 49 页: 关键代码: 提示词与输出解析	13
6 总结	13
第 50 页: 最终系统做到了什么	13
第 51 页: 限制与下一步	13
第 52 页: Thanks	13

使用说明

这份文档按当前 Beamer 的 52 页逐页编写。每一页的讲稿不是逐字背诵稿，而是课堂汇报时可以照着展开的讲解逻辑：先讲这页解决什么问题，再讲技术细节，最后点出这一页和整个系统闭环的关系。涉及英文函数名、文件名和指标名时，旁边都补了中文解释，避免听众只看到代码名字但不知道它负责什么。

1 开场与系统总览

第 1 页：MEMO-Appflow

讲稿 这一页先把项目定位讲清楚：MEMO-Appflow 不是一个普通的应用推荐列表，而是一个在 Android 端侧运行的系统。它从两个方向获取信息：一边观察真实 app 使用序列，一边通过 eBPF 观察操作系统内部发生了什么。然后本地 MAPLE 大模型把这些证据合在一起，输出接下来可能使用的 Top-3 应用，以及系统应该执行的调度动作。

可以这样开场：我们做的是从 **app + OS 到 app + OS** 的闭环。输入端既有 app 行为，也有操作系统证据；输出端既给用户 app 推荐，也让系统执行预热、内存整理、服务刷新等调度动作。这样项目目标就不是“猜下一个 app”这么窄，而是用推理改善手机使用体验和系统资源状态。

提示 标题页只讲核心定位，不展开所有细节。重点强调讲解人 Jingyi Guo，组员 Mosha Huang、Kewei Chen、Chengyu Fan，以及日期 2026 年 7 月 2 日。

第 2 页：汇报路线

讲稿 这一页说明整场汇报的结构。先解释为什么只看 app 序列不够，然后讲 eBPF 采集和数据压缩，接着讲 MAPLE 如何推理，再讲预测如何落到真实 Top-3 应用和系统动作，最后看实验与结果。

可以提醒听众：后面不是按代码目录讲，而是按产品链路讲。每一段都回答一个问题：我们收集了什么、为什么这样整理、大模型拿到了什么、输出如何变成手机上的真实行为、实验如何证明这些设计有用。

第 3 页：项目不是单纯的 app 推荐

讲稿 这一页要纠正一个常见误解：如果只用历史 app 序列预测下一个 app，系统很容易变成一个浅层统计模型。我们的目标更深，是理解用户当前行为背后的系统状态。比如同样打开聊天软件，背后可能是单纯聊天、发送图片、准备扫码、接收视频流，这些场景在 app 名字上都很像，但在网络、相机、显示、Binder 服务上会有不同信号。

所以 MEMO-Appflow 把 app 行为和 OS 证据一起放进 MAPLE：app 序列提供“用户在做什么”的显式线索，eBPF 和系统状态提供“系统正在承受什么压力、哪些服务被调动”的隐式线索。最后输出也分两类：Top-3 app 推荐面向用户，系统调度面向 Android 资源管理。

第 4 页：宏观闭环：系统观察、模型决策、系统执行

讲稿 这一页讲完整闭环。第一步是观察：实时采集前台 app、eBPF tracepoint 事件、内存、电池、网络、显示、服务等状态。第二步是决策：把这些信息压缩成 MAPLE context，让本地模型推理。

第三步是执行：更新桌面 Widget，并根据 scheduler bits 执行预先编程好的系统动作。

这里可以强调“回到系统”很重要。很多 demo 到模型输出就结束了，但我们的系统会把模型输出解析为具体动作，例如是否 warm launch 候选 app、是否减少预热、是否做内存 trim、是否刷新服务管理器信息。模型不是直接执行任意 shell 命令，而是在我们定义好的动作集合里选择 0/1，这样更可控。

第 5 页：手机内运行边界

讲稿 这一页讲部署边界：Host 电脑只负责安装 APK、推送模型和查看日志，产品逻辑不依赖电脑。手机或 emulator 内部自己完成采集、解析、压缩、推理、调度和 Widget 展示。

这个边界非常关键，因为我们最终要迁移到真机。如果 pipeline 依赖电脑上的 Python 或 adb 脚本，就不是一个真正的端侧产品。现在的设计把数据流留在 Android 内：Root/native collector 负责底层采集，Kotlin 负责结构化和调度，MAPLE 本地模型负责推理，Widget 负责给用户入口。

2 eBPF 采集与压缩

第 6 页：为什么要引入 eBPF

讲稿 这一页讲 eBPF 的作用。eBPF 是 Linux 内核提供的安全可加载程序机制，可以在内核事件发生时记录信息。我们用它不是为了炫技，而是为了拿到 app 层看不到的证据：进程执行、网络收发、Binder 服务、内存 reclaim、显示相关事件等。

只看 app 序列会丢掉很多系统层含义。比如“打开浏览器”和“打开视频 app”都可能产生网络流量，但 UDP recvfrom、display refresh、MediaCodec 相关信号可以帮助区分用户是在加载网页、看视频，还是只是后台同步。eBPF 的价值是让模型看到这些深层证据。

第 7 页：采集的 14 个 tracepoints

讲稿 这一页列出我们采集的 tracepoints。tracepoint 是 Linux 内核中相对稳定的事件点，比直接挂不稳定 kprobe 更适合 ARM/Android 兼容。我们覆盖的方向包括进程、网络、内存、调度、系统调用等。

讲的时候可以把它分成几类：进程类用于观察 app 或服务启动；网络类用于观察 sendto、recvfrom 这样的收发行为，尤其 UDP；内存类用于观察 reclaim、page fault、内存压力；调度类用于观察系统忙不忙；系统调用和服务类用于补充 Binder、文件和设备访问证据。每一条 tracepoint 都不是单独给模型看，而是先聚合成 counter key。

第 8 页：Pixel 5 上的 kernel/runtime 条件

讲稿 这一页说明真机运行需要什么条件。eBPF 不是普通应用权限能做的事情，通常需要 root、内核打开 BPF 相关配置、BTF 或可用 tracefs/debugfs，以及能运行 native collector。Pixel 5 的环境用于验证这些条件。

这里要实事求是：真机部署不是所有 Android 手机都天然支持。我们当前的产品形态面向已 root 的实验设备，依赖 Linux/Android 内核能力。这个限制不影响研究结论，因为我们的研究目标是验证 app + OS 证据驱动的端侧推理和调度闭环。

第 9 页：tracepoint 到 counter key

讲稿 这一页讲“为什么要计数”。eBPF 原始事件非常多，如果每一行都送给模型，prompt 会爆炸，而且大量重复事件对语义帮助有限。于是我们把 tracepoint 事件转成 counter key，也就是“某类事件在某个窗口内出现了多少次”。

表格的读法是：左边是内核事件来源，中间是我们抽象出的事件类型，右边是最后进入 context 的计数键。比如多次 UDP recvfrom 不会重复写几百行，而会变成 `network.recvfrom.udp = 多少次`。这样模型看到的是高密度证据：网络收包明显升高、display 刷新明显升高、内存 reclaim 明显升高，而不是一堆肉眼难读的重复 trace。

第 10 页：真实采集窗口长什么样

讲稿 这一页展示真实窗口，不是手写样例。一个窗口内会有 app 事件、eBPF 事件和系统状态快照。它们都有时间戳，因此后面可以按时间对齐。

讲的时候可以说：这一页的原始记录看起来会比最终 JSON 更杂，因为它接近系统真实输出。我们的产品不会把这样的原始格式直接丢给用户，也不会原封不动丢给模型，而是在 Android 端先解析、去重、计数、分类。保留真实窗口的意义是证明证据来源是自然产生的系统行为。

第 11 页：Android 端采集运行时：代码结构概览

讲稿 这一页讲运行时结构。Android 侧不是调用电脑脚本，而是通过 root bridge 或 root shell 启动 native collector。collector 在手机本地 attach tracepoints，把事件写到本地文件或管道；Kotlin 服务再读回这些事件，进入解析和压缩流程。

讲图里的每个节点：EBPFCollectorService 是前台服务入口，负责接收按钮、广播和实时任务；DeviceCollectorDeployer 确保手机上有可执行 collector；memo_libbpf_collector 是 native 采集器，负责 attach 内核 tracepoint 并写出 MEMO_* 行；前台 app 采样器同步记录当时屏幕上是谁；窗口结束后停止 collector、读取 trace；EBPFTraceParser 把文本行变成 Kotlin 事件对象，交给后面的窗口压缩和 MAPLE context 构造。

第 12 页：app 序列和 eBPF 的时间对齐

讲稿 这一页讲为什么必须对齐。MAPLE 不能只看到“最近打开过哪些 app”，也不能只看到“最近网络和内存事件很多”。它需要知道这些事情是否发生在同一时间窗口里。

例如某一分钟内前台 app 是聊天软件，同时 UDP recvfrom、Binder、camera 服务访问增加，这和“聊天后准备拍照/发图”的场景更一致。如果 eBPF 事件发生在更早的后台同步阶段，就不应该强行解释成当前 app 行为。所以我们用时间窗口把 app sequence 和 OS evidence 对齐，再构造 timeline。

第 13 页：timeline window 示例

讲稿 这一页展示一个时间窗口示例。它不是为了给模型看完整流水账，而是告诉模型：在这个窗口里，前台 app 是什么，系统证据出现了哪些类别，每类出现多少次，以及系统状态怎样变化。

可以逐行解释：foreground app 表示窗口内主要使用的真实应用；network/sendto/recvfrom 表示网络活跃；display 表示界面刷新或渲染相关信号；memory 表示 reclaim、available memory 等

状态。模型用这些上下文判断下一步可能的用户意图和调度策略。

第 14 页：eventType 到 category：权重矩阵

讲稿 这一页讲权重矩阵。权重矩阵就是把底层事件类型映射到更高层语义类别的规则表。比如网络事件对 communication、browser、media streaming 类别更有贡献；camera/media 事件对 camera、gallery、video 类别更有贡献；display/render 事件对 scrolling、video、UI-heavy 类别更有贡献。

表格的每个数字可以理解为“这个事件类型支持这个类别的强度”。它不是最终预测模型，而是给 MAPLE 准备高层语义特征。这样做的好处是模型不用从几千个底层 counter 里自己猜语义，而是先看到更稳定、更短的 category score。

第 15 页：Category score 如何算

讲稿 这一页讲计算方式。Category score 是“语义强度分”，不是概率。每个窗口里先统计各类 eBPF counter，然后乘以权重矩阵里的分数，最后用 cap 截断。

用 Network IO 例子讲：如果 SENDTO 120 次、RECVFROM 180 次，且二者对 Network 的权重都是 3，那么 raw score 是 900；但 Network 是高频事件，不能让它淹没 Binder、Memory、Input，所以用 cap 把分数截到 420。还要讲清楚：Memory Management、Android Service IPC、App Process Runtime 这些 OS-only 类别不能作为“下一个 app”，但能影响调度动作，所以 context 里把 app prediction candidates 和 scheduler evidence 分开。

第 16 页：Context JSON 字段

讲稿 这一页讲 MAPLE 输入字段。Context JSON 不是随便拼的，它要同时保留 app 层、OS 层和调度层信息。

字段可以这样解释：app_timeline 是真实 app 使用序列；ebpf_counters 是压缩后的内核证据；category_scores 是语义类别分数；system_state 是内存、电池、网络等状态；memory_ema 是长期有界记忆；recent_window 是最近窗口。考虑这些字段是为了让模型同时回答两个问题：接下来可能用什么 app，以及当前系统该如何调度。

第 17 页：真实 Context JSON 摘录

讲稿 这一页展示真实输入片段。代码框里可以带注释讲：这个 JSON 已经不是原始 trace，而是给模型看的结构化证据。

重点是让听众看到 MAPLE 拿到的不只是 eBPF，也包括 app 信息。app 序列告诉模型用户显式行为；eBPF counters 告诉模型系统内部反应；system state 告诉模型当前资源压力；scheduler context 告诉模型哪些动作可选。这样模型输入既紧凑又包含多源证据。

第 18 页：压缩不是丢证据，而是换表达

讲稿 这一页回应“压缩会不会损失信息”的问题。压缩确实会丢掉每一条重复事件的细节，但保留了模型最需要的语义：某类事件在某个时间窗口内是否显著升高，以及和 app 行为是否同时发生。

为什么要这样做？因为端侧小模型上下文有限，raw trace 太长会让 prompt 变慢、变贵、变不稳定。计数压缩把重复事件变成“事件类型 + 次数 + 时间窗口”，既保留强度，又减少 token。对预测和调度来说，这比把上千行重复 trace 直接塞给模型更有效。

3 MAPLE 推理与真实应用映射

第 19 页：MAPLE 三阶段

讲稿 这一页讲推理分阶段。Stage 1 让模型根据 context 判断用户可见的 app 类别；Stage 2 把类别转换成候选 App ID；Stage 3 输出 scheduler bits，告诉系统哪些调度动作要执行。

分阶段的原因是小模型直接输出“Top-3 app + 调度动作 + 解释”容易格式混乱。拆开以后每一步任务更清楚：先理解场景，再给应用候选，再给动作 bitmask。这样 parser 更容易校验，也更适合端侧小模型。

第 20 页：Stage 1 prompt：预测用户可见 app 类别

讲稿 这一页讲 Stage 1 的 prompt。这里要特别强调 prompt 的目标不是预测进程名，不是 SurfaceFlinger，也不是 Binder 服务，而是预测用户接下来可能使用的真实应用类别。

可以解释：我们在 prompt 里明确写“请预测用户接下来要使用的三个 app 类别”，并且把 OS 证据作为辅助证据。比如网络活跃不等于推荐“network process”，而是帮助判断用户可能继续用聊天、浏览器、视频或支付类应用。

第 21 页：Stage 1 真实输出

讲稿 这一页展示模型真实输出。排版上已经把 source 和解释放到下方，讲解时不要只念结果，要解释每个字段代表什么。

例如输出里的类别排序表示模型认为哪些用户场景更可能发生；confidence 或 score 表示模型对该类别的相对确信程度；解释文本说明它用了哪些证据。这里要指出：小模型有时不够听话，输出格式可能脏，所以后面需要 parser 做容错，但核心语义仍然能被提取出来。

第 22 页：Stage 2 prompt：类别到候选 App ID

讲稿 这一页讲 Stage 2。模型先输出类别还不够，因为用户最终要看到可点击的真实 app。Stage 2 把类别转成候选 App ID，例如 communication、camera、browser、media 等类别对应一组内部 ID。

这里的 App ID 不是 Android 包名，而是 MAPLE 内部的候选编号。它的作用是把大模型输出控制在稳定集合里，便于后续解析和映射。真正显示给用户前，还要通过 AppIdMapping 扫描手机安装应用，把候选 ID 映射到真实包名、应用名和图标。

第 23 页：Stage 3: scheduler_bits 动作协议

讲稿 这一页讲大模型如何赋能系统调度。我们不是让模型随便写命令，而是提前定义一组动作，每个动作对应一个 bit。模型输出一串 0/1，1 表示执行，0 表示不执行。

这种 bitmask 设计的好处是安全和可控。比如 bit 0 可以表示 Top-1 warm launch, bit 1 表示 memory trim, bit 2 表示 service manager refresh, bit 3 表示降低预热激进度。模型只负责判断“当前场景是否需要这个动作”，具体怎么执行由 Android 端代码实现。

第 24 页：小模型输出解析也要工程化

讲稿 这一页讲 parser。端侧小模型能力有限，可能输出多余文字、换行不规范、字段名不一致，甚至没有严格 JSON。我们的 parser 负责从脏输出里提取可用字段，比如 Top-3、App ID、scheduler bits、解释文本。

这页的重点是工程可靠性。研究系统不能假设模型每次完美遵守 prompt，所以我们必须写解析、校验和边界处理。当前 parser 能处理常见脏输出，但后面仍需要更 robust，这是限制与下一步里会再次提到的点。

第 25 页：从 MAPLE App ID 到真实 Top-3：映射表怎么来

讲稿 这一页讲映射不是硬编码。MAPLE 输出的是语义候选，例如 Communication 或 App 110，不是固定包名。Android 端用 PackageManager 扫描本机所有可启动应用，读取应用名、包名、图标、launch intent、权限、默认角色、intent 能力和 Android category，再给每个 app 自动打语义标签。

排序规则大致是：默认角色和系统 intent 能力权重最高，权限和 Android category 次之，包名/label 关键词再往后，最后再结合最近使用和是否用户安装。这样做的意义是：系统没有写死“Communication = QQ”。QQ 只是这台手机上被扫描出来且匹配 Communication 的一个候选；换机后候选池会从新手机的 app 重新生成。

第 26 页：从 MAPLE App ID 到真实 Top-3：用户看到什么

讲稿 这一页展示最终用户视角。用户不会看到 eBPF、Binder、SurfaceFlinger 这些系统术语，而是看到 Top-3 真实应用图标、应用名和更新时间。

这里重点解释迁移性：如果手机 A 装了 QQ，Communication 候选可能是 QQ、Messages、Phone；如果手机 B 没有 QQ 但装了微信，扫描器会根据微信的 label、包名、权限、intent 能力和通信类特征把它放进 Communication 候选池，所以 Top-3 会自然落到 WeChat、Messages、Phone。模型不需要“认识微信”，它只判断语义，具体 app 由本机扫描结果决定。

第 27 页：ActionExecutor 执行的是真动作

讲稿 这一页讲 ActionExecutor。它把 MAPLE 输出的 scheduler bits 转成 Android 端动作，不是只打印日志。动作包括 Widget 更新、Top-1 warm launch、内存 trim、服务刷新、根据压力降低预热激进度、根据网络和 camera/media 证据调整候选优先级等。

这里要强调两点。第一，动作是预先编程好的，模型不能任意操作系统。第二，动作会结合系统状态执行，比如内存压力高时不会激进预热，display/UI 压力高时会减少可能造成卡顿的动作。这体现了“模型推理 + 系统约束”的设计。

第 28 页：实时模式任务队列

讲稿 这一页讲实时产品逻辑。系统持续收集数据，每 3 分钟形成一个窗口，然后异步触发 MAPLE 推理和调度。推理时采集不中断，下一窗口仍然继续收集。

可以解释流水线：窗口 A 采集结束后进入解析和推理，同时窗口 B 已经开始采集。这样不会变成“收完停下来推理，推完再收”的串行系统。为了防止后台线程爆炸，我们限制同时运行的推理任务数量，维护有界 memory，并对过期任务做状态管理。

第 29 页：Widget：把预测变成可用的产品入口

讲稿 这一页补充 Widget 的产品意义。Widget 是桌面上的入口，让用户不用打开 MEMO app 也能看到最新 Top-3 推荐，并直接点击打开。

这和普通 debug 页面不同。debug 页面给开发者看证据和日志，Widget 给真实用户使用。它把模型预测变成日常手机交互的一部分，也能展示“最近更新时间”，让用户知道后台实时模式正在工作。

4 实验设计与结果

第 30 页：实验总览：我们要回答什么

讲稿 这一页开始进入实验部分。先提出 research questions，而不是直接甩数字。我们主要回答三类问题：实时闭环能不能连续运行；eBPF 信息是否真的让预测更好；系统调度是否能改善手机压力指标。

这样安排实验更清楚：9 分钟实时实验验证产品闭环；理论消融和真实 30 条预测实验验证 eBPF 对预测有用；性能 A/B 验证调度动作是否对系统状态有帮助。不同实验回答不同问题，不混在一起。

第 31 页：实验之间的关系

讲稿 这一页说明几个实验不是互相替代，而是互相支持。理论消融先在可控数据上看加入 eBPF 后是否改善预测；真实记录预测实验再看真实采集数据上是否有同样趋势；9 分钟实验证明实时窗口和调度能跑；性能 A/B 关注系统压力变化。

可以特别说：理论消融和真实消融的前后结果具有一致性，也就是 eBPF 证据确实提升了 app-only baseline 之外的信息量。这样可以避免听众误以为消融只是孤立数字。

第 32 页：9 分钟实时实验：动机与方案

讲稿 这一页讲为什么做 9 分钟。因为实时模式每 3 分钟推理一次，9 分钟可以覆盖三个完整窗口，足够观察 Top-3 是否随着使用序列更新，以及 scheduler bits 是否在不同窗口触发动作。

实验方案是设计一个较极端但合理的使用序列，让前几个窗口偏社交/网络，后面窗口偏 camera/media 或浏览/显示压力。系统持续采集 app 序列和 eBPF，窗口结束后进入 MAPLE 推理，输出 Top-3 和调度动作。整个过程按真实实时模式运行，不是离线手改 JSON。

第 33 页：9 分钟实时实验：结果

讲稿 这一页讲结果。重点看每个 3 分钟窗口的 Top-3、MAPLE 耗时、输入证据摘要、scheduler bits 和执行动作。

讲解时要避免只说“变了”。应该说变化来自什么证据：比如前一窗口网络和聊天类 app 更强，所以推荐偏通信/浏览；后一窗口 display、camera/media 证据更强，所以推荐转向相机、相册或媒体应用。这样说明 Top-3 不是随机变化，而是由 app + OS evidence 共同驱动。

第 34 页：9 分钟实验中的调度记录

讲稿 这一页展示调度日志。日志里要看三件事：模型输出了哪些 scheduler bits；ActionExecutor 解析出了哪些动作；这些动作是否被实际执行或因系统约束被降级。

例如 Top-1 warm launch 表示系统尝试轻量预热最可能使用的应用；memory trim 表示系统尝试缓解内存压力；service manager refresh 表示刷新系统服务状态；display/UI policy 表示避免在界面压力高时做过重动作。这页证明模型输出已经进入系统动作层。

第 35 页：30 条真实记录预测实验：实验设计

讲稿 这一页讲预测实验怎么做。我们用 3 类 persona，也就是使用风格：media、productivity、social。每类 10 条，共 30 条。使用意图脚本和 ground truth 是 Codex 根据 persona 生成的；但执行阶段不是手写 trace，而是 Android runner 在手机上真实启动 app、发 input 命令，并自然采集 app timeline 和 raw eBPF。

为了有 ground truth，我们把脚本后半段将要使用的 Top-3 app mask 掉，只把前半段上下文给模型预测，再用被 mask 的部分评分。比较对象有两个：app-only baseline 只看 app 序列；full MAPLE 看 app 序列、eBPF counters、category scores 和 system state。这个实验的研究问题是：加入 eBPF 是否让 Top-3 预测更接近 ground truth。

第 36 页：30 条预测实验：评估指标

讲稿 这一页讲指标。Top-3 accuracy 表示预测的三个应用里是否命中任一 truth；precision 表示预测列表里有多少比例是正确的；recall 表示 truth 里有多少比例被预测覆盖；F1 是 precision 和 recall 的调和平均。

数学上，如果预测集合是 P ，真实集合是 T ，那么 precision 是 $|P \cap T|/|P|$ ，recall 是 $|P \cap T|/|T|$ ，F1 是 $2PR/(P + R)$ 。Top-3 accuracy 更像“有没有猜中至少一个”，precision/recall/F1 更细地衡量预测集合质量。

第 37 页：30 条预测实验：单条指标怎么算

讲稿 这一页用单条样本直观解释。现在 slide 上的例子是预测 QQ、Tencent Meeting、Chrome，truth 是 QQ、Chrome、Camera，交集有 QQ 和 Chrome 两个。Precision@3 是 2/3，Recall@3 是 2/3，F1 也是 2/3；Top-3 Hit 是 1，因为至少命中一个；MRR 是 1，因为第一个预测 QQ 就在 truth 里。

这页的价值是让听众知道指标不是黑盒。我们的代码就是在每条样本的预测列表和 truth 列表上做集合匹配，再对 30 条样本求平均。这样后面看到总体准确率和 F1 时，能知道它们怎么来的。

第 38 页：理论消融：先在可控数据上验证 eBPF 信号

讲稿 这一页讲理论消融放在真实消融之前。理论消融使用更可控的构造数据，逐步移除 network、camera/display、binder/memory 等证据，观察预测指标下降。

这不是最终产品实验，而是先验证一个机制：eBPF 的不同证据通道确实携带有用信息。后面真实 30 条实验在真实采集记录上得到一致趋势，说明这个结论不是只在理论数据里成立。

第 39 页：30 条真实记录：最终预测结果

讲稿 这一页展示最终预测结果。讲的时候关注 app-only baseline 和 full MAPLE 的差异：如果 full MAPLE 的 Top-3 accuracy、precision、recall、F1 更高，就说明 app + OS 多源上下文比单纯 app 序列更有信息。

还要提醒：source 放在页面底部，听众可以回到结果文件查看每条样本预测、truth、输入上下文和指标。这样结果不是口头数字，而是可以追溯到具体记录。

第 40 页：单条样本的直观结果

讲稿 这一页用一条真实样本解释模型为什么预测这些 app。

这条样本是 media_browsing 风格：前半段在媒体/浏览场景中切换，设备真实采集到网络收发、Binder、显示刷新和 app 切换证据。被 mask 的未来动作是 QQ、Tencent Meeting、Chrome。语义上看，这像是媒体浏览后继续进入通信、会议或网页，所以 full path 把这三个都排出来是合理的。这里要讲一点故事，不要只念指标：用户刚才的行为和 OS 证据共同说明“接下来仍然会在通信/网络相关 app 里移动”。

第 41 页：系统调度性能实验：Stage 3 是否有用

讲稿 这一页讲系统调度是否真正改善性能。这里不用旧的“MEMO 开/关压力 A/B”，而是改成更直接的 Stage 3 调度实验：先用真实 eBPF 和 MAPLE 选出同一个 Top-1 目标 app，然后 baseline 和 Stage 3 都测这个同一个 app。

实验设计要讲清楚：MAPLE 在真实 eBPF scenario 上选出的 Top-1 是 QQ。模型输出的调度开关是：

```
scheduler_bits=101010100
```

Baseline 分支先 force-stop QQ，然后不执行调度，直接冷启动 QQ 并跑同一段动作序列。Stage 3 分支也先 force-stop QQ，但随后执行 MAPLE 选择的调度动作，再启动同一个 QQ。这样比较才公平，因为测量目标一致，差别集中在“有没有执行 Stage 3 scheduler bits”。

结果解释：Launch TotalTime 从 433 ms 降到 168 ms，改善 61.20%；WaitTime 从 440 ms 降到 173 ms，改善 60.68%；综合压力分数从 422.56 降到 9.71，改善 97.70%。CPU busy、iowait、reclaim、jank rate 也下降。这里的核心结论不是“所有场景永远提升”，而是这次真实 Pixel 5 实验表明：当 Stage 3 的 Top-1 warm launch、memory trim 和 service manager refresh 确实作用到同一个被测 app 时，系统指标可以明显改善。

还要强调动作记录：Stage 3 不是只输出文本。日志里有 7 条动作记录，包括 widget update、latency policy、scheduler plan、Top-1 warm launch、memory trim、drop cache safety skip、service

manager refresh。drop cache 被跳过不是失败，而是安全检查生效：内存不是 critical，所以不执行风险更高的 cache drop。

提示 这一页讲的时候要把“同一个目标 app”说清楚，因为这是这次重构实验设计的关键。结论只围绕本次 v2 结果展开，不再引用旧手机压力 A/B 的平均值。

5 关键代码与实现落点

第 42 页：关键代码：三分钟采集窗口

讲稿 这一页讲 `RealtimeWindowRunner.kt`。这个文件负责实时模式的核心调度：持续采集，按 3 分钟切窗口，窗口结束后触发解析、MAPLE 推理、动作执行和 Widget 更新。

可以解释“Runner”就是后台任务调度器。它保证推理和采集以流水线方式运行，避免推理时停止收集；同时也限制并发，防止后台线程越来越多导致手机压力变大。

第 43 页：关键代码：有界实时记忆

讲稿 这一页讲 `RealtimeMemoryStore.kt`。它维护历史信息的 EMA，也就是指数衰减平均。新窗口权重重大，旧窗口权重逐渐变小。

为什么需要有界 memory？因为实时模式长期运行，不能把所有历史窗口都塞进 prompt。EMA 保留长期偏好，例如用户近期更常用通信或视频类应用，同时不会让 memory 无限增长。这就是“有记忆但不爆炸”的设计。

第 44 页：关键代码：MAPLE 输入构造

讲稿 这一页讲 `MapleScenarioBuilder.kt`。它把 app timeline、eBPF counters、category scores、system state 和 EMA memory 合成一个 MAPLE scenario。

这个文件是 app + OS 融合的核心位置。没有它，采集到的东西只是散乱日志；有了它，模型拿到的是结构化上下文。讲的时候可以说：这一步决定 MAPLE “看见什么”，所以它直接影响预测质量。

第 45 页：关键代码：本地模型调用

讲稿 这一页讲 `MapleShellBackend.kt`。它负责在 Android 端调用本地 MAPLE 引擎和 GGUF 模型，把 scenario JSON 送进去，再读回模型输出。

这里可以解释本地模型的意义：推理不依赖云端，隐私更好，也符合端侧产品定位。但本地小模型能力有限，所以 prompt 要短，输出解析要容错，推理要异步放在后台。

第 46 页：关键代码：真实应用映射

讲稿 这一页讲 `AppIdMapping.kt`。它使用 PackageManager 扫描手机上的应用，建立类别、App ID 和真实包名之间的映射。

用户看到的 Top-3 是真实 app 名称和图标，这一步很关键。否则模型输出可能停留在 App 110 或 communication 这种内部概念，用户无法理解也无法点击。

第 47 页：关键代码：系统调度动作

讲稿 这一页讲 `ActionExecutor.kt`。它是 MAPLE 输出进入系统动作的入口。

讲解时可以按动作类型说：推荐更新属于产品展示动作；warm launch 属于预热动作；memory trim 属于内存压力响应；service manager refresh 属于系统服务状态刷新；网络、camera/media、display/UI 策略属于候选优先级和降级策略。这样听众能明白每个动作不是抽象词，而是对应代码路径。

第 48 页：关键代码：桌面 Widget

讲稿 这一页讲 `MemoWidgetProvider.kt`。Widget 负责把 Top-3 推荐显示到桌面，并让图标可点击打开应用。

这里要强调 Widget 是产品的一部分，不是装饰。实时模式每 3 分钟更新一次推荐，Widget 让用户不用进入 MEMO app 就能得到预测结果，这才体现推荐对使用体验的帮助。

第 49 页：关键代码：提示词与输出解析

讲稿 这一页讲 `prompt_builder.cpp` 和 `result_parser.cpp`。前者构造给 MAPLE 的输入提示词，后者把模型输出解析成可执行结构。

prompt 要明确“预测用户接下来要使用的三个 app”，还要要求输出 scheduler bits。parser 要从模型输出里拿到 Top-3、解释和 bitmask。小模型不够听话时，这两个文件就是稳定性的关键。

6 总结

第 50 页：最终系统做到了什么

讲稿 这一页总结成果。系统已经形成端侧闭环：真实采集 app + OS 证据，压缩成 MAPLE context，本地模型推理，输出 Top-3 和 scheduler bits，再映射为真实应用和系统动作，最后通过 Widget 和后台调度影响手机体验。

可以用一句话收束：我们不是只做一个 app 推荐模型，而是把 Android 的应用层信息和操作系统层信息结合起来，再反过来服务应用推荐和操作系统调度。

第 51 页：限制与下一步

讲稿 这一页讲限制要诚实但聚焦。第一，小模型能力有限，有时不够听话，输出格式不一定完全遵守 prompt；虽然有 parser 处理脏输出，但还不够 robust。第二，实时性和轻量性有 tradeoff：系统一直后台运行，每 3 分钟推理一次，实时性更强，但开销也更高。

下一步可以讲三点：更细粒度的 scheduler policy，让每个 bit 对应更准确的系统动作；更强的 app mapping 语义分类，让不同设备上的真实 app 映射更准；更自动化的能力检测和部署脚本，让 root、kernel、BTF、collector、模型路径这些条件更容易检查和配置。

第 52 页：Thanks

讲稿 最后一页收尾。可以说：今天的核心贡献是把 MEMO-Appflow 从 host 脚本原型推进到 Android 端侧闭环，并且让 app + OS 证据共同驱动用户可见的 Top-3 推荐和系统可执行的调度动

作。

最后可以邀请老师和同学提问，尤其是 eBPF 证据、MAPLE prompt、scheduler bits、Widget 产品化和实验指标这几部分。